

Week 3 Structured Notes: Types of Recursion

1. Week 3 Main Goal

The goal of Week 3 is to understand the main types of recursion.

In Week 1, you learned the basic idea of recursion:

- A function can call itself.
- A recursive function needs a base case.
- Work can happen during the calling phase or returning phase.

In Week 2, you learned how recursion works in memory:

- Each recursive call creates a stack frame.
- Local variables are separate for each call.
- Static and global variables are shared.
- Simple recursion usually uses $O(n)$ time and $O(n)$ stack space.

In Week 3, you learn how to look at recursive code and identify the pattern.

By the end of Week 3, you should be able to identify these five types of recursion:

1. Tail recursion
2. Head recursion
3. Tree recursion
4. Indirect recursion
5. Nested recursion

The most important Week 3 skill is classification. That means you should be able to look at code and say what type of recursion it is, and why.

2. Quick Revision Before Week 3

Before learning the types of recursion, remember these basic ideas.

2.1 Recursion

Recursion means a function calls itself.

Example:

```
void fun(int n) {  
    if (n > 0) {  
        fun(n - 1);  
    }  
}
```

Here, `fun()` calls itself, so it is a recursive function.

2.2 Base Case

The base case is the condition that stops recursion.

Example:

```
if (n > 0)
```

When `n` becomes `0`, the condition becomes false. Then the function stops calling itself.

2.3 Recursive Case

The recursive case is the part where the function calls itself again.

Example:

```
fun(n - 1);
```

Here, the input becomes smaller each time. This helps the function move toward the base case.

2.4 Calling Phase

The calling phase happens when recursive calls are going deeper.

Example:

```
printf("%d ", n);  
fun(n - 1);
```

Here, printing happens before the recursive call. So the output appears while recursion is going down.

2.5 Returning Phase

The returning phase happens when recursive calls finish and return back one by one.

Example:

```
fun(n - 1);  
printf("%d ", n);
```

Here, printing happens after the recursive call. So the output appears while recursion is returning back.

3. Why Week 3 Is Important

Many beginners can write a recursive function, but they cannot explain what type of recursion it is.

That is a problem.

If you understand the type of recursion, you can understand:

- Where the main work happens
- How the function will execute
- Whether it is easy to convert into a loop
- Whether it may become slow
- Whether the trace will be simple or difficult

Week 3 teaches you to recognize recursive patterns instead of memorizing code.

Simple Week 3 question:

Where is the recursive call placed, and how many recursive calls are made?

Simple answer:

The position and number of recursive calls decide the recursion type.

4. Tail Recursion

4.1 Meaning of Tail Recursion

Tail recursion means the recursive call is the last operation in the function.

In simple words:

The function does its work first, then calls itself. After the recursive call, there is no more work left.

Example

```
#include <stdio.h>

void tail(int n) {
    if (n > 0) {
        printf("%d ", n);
        tail(n - 1);
    }
}

int main() {
    tail(3);
    return 0;
}
```

Output

```
3 2 1
```

4.2 Why This Is Tail Recursion

Look at the important part:

```
printf("%d ", n);
tail(n - 1);
```

The recursive call is:

```
tail(n - 1);
```

This is the last statement inside the `if` block.

There is no work after this recursive call.

So this is tail recursion.

4.3 Dry Run of tail(3)

Function call:

```
tail(3);
```

Calling phase:

```
tail(3): print 3, then call tail(2)
tail(2): print 2, then call tail(1)
tail(1): print 1, then call tail(0)
tail(0): stop
```

Final output:

```
3 2 1
```

The printing happens before the recursive call, so output happens during the calling phase.

4.4 Tail Recursion and Loop Conversion

Tail recursion is usually easy to convert into a loop.

Recursive version:

```
void tail(int n) {
    if (n > 0) {
        printf("%d ", n);
        tail(n - 1);
    }
}
```

Loop version:

```
void tailLoop(int n) {
    while (n > 0) {
        printf("%d ", n);
        n--;
    }
}
```

```
}  
}
```

Both versions print:

```
3 2 1
```

Why is tail recursion easy to convert?

Because nothing is waiting after the recursive call.

The function does the work, changes the input, and repeats.

That is similar to how a loop works.

4.5 Time and Space Complexity of Tail Recursion

For `tail(n)`:

- Time complexity: `O(n)`
- Space complexity: `O(n)` in normal recursion

Why?

The function makes about `n + 1` calls, and each call creates a stack frame.

However, some compilers may optimize tail recursion and reduce stack usage. But as a beginner, assume tail recursion uses `O(n)` stack space unless optimization is clearly mentioned.

5. Head Recursion

5.1 Meaning of Head Recursion

Head recursion means the recursive call happens before the main work.

In simple words:

The function calls itself first. After the smaller calls finish, the function does its work while returning.

Example

```
#include <stdio.h>

void head(int n) {
    if (n > 0) {
        head(n - 1);
        printf("%d ", n);
    }
}

int main() {
    head(3);
    return 0;
}
```

Output

```
1 2 3
```

5.2 Why This Is Head Recursion

Look at the important part:

```
head(n - 1);
printf("%d ", n);
```

The recursive call happens first:

```
head(n - 1);
```

The work happens after the recursive call:

```
printf("%d ", n);
```

So this is head recursion.

5.3 Dry Run of head(3)

Function call:

```
head(3);
```

Calling phase:

```
head(3) calls head(2)
head(2) calls head(1)
head(1) calls head(0)
head(0) stops
```

Returning phase:

```
head(1) prints 1
head(2) prints 2
head(3) prints 3
```

Final output:

```
1 2 3
```

The function goes down first without printing. Then it prints while returning.

5.4 Head Recursion and Loop Conversion

Head recursion is harder to convert directly into the same loop structure.

Recursive version:

```
void head(int n) {
    if (n > 0) {
        head(n - 1);
        printf("%d ", n);
    }
}
```

Output for `head(3)`:

1 2 3

If you write a simple loop like this:

```
void wrongLoop(int n) {  
    while (n > 0) {  
        printf("%d ", n);  
        n--;  
    }  
}
```

Output:

3 2 1

This is not the same output.

To match head recursion, the loop must count upward:

```
void headLoop(int n) {  
    int i = 1;  
    while (i <= n) {  
        printf("%d ", i);  
        i++;  
    }  
}
```

Output:

1 2 3

Why is head recursion harder to convert?

Because the work happens after returning from recursion. A normal loop usually works immediately in forward execution.

5.5 Time and Space Complexity of Head Recursion

For `head(n)` :

- Time complexity: `O(n)`
- Space complexity: `O(n)`

Why?

There is one recursive call per level, and the stack grows until the base case is reached.

6. Difference Between Tail Recursion and Head Recursion

Feature	Tail Recursion	Head Recursion
Recursive call position	Last operation	Before main work
Work happens	Before recursive call	After recursive call
Main phase	Calling phase	Returning phase
Example output for n = 3	<code>3 2 1</code>	<code>1 2 3</code>
Loop conversion	Easier	Harder to convert directly
Stack use normally	<code>O(n)</code>	<code>O(n)</code>

Simple memory trick:

- Tail recursion speaks while going down.
 - Head recursion speaks while coming back up.
-

7. Tree Recursion

7.1 Meaning of Tree Recursion

Tree recursion means a function calls itself more than once.

In simple words:

One function call creates more than one recursive branch.

Example

```
#include <stdio.h>

void tree(int n) {
    if (n > 0) {
        printf("%d ", n);
        tree(n - 1);
        tree(n - 1);
    }
}

int main() {
    tree(3);
    return 0;
}
```

Output

```
3 2 1 1 2 1 1
```

7.2 Why This Is Tree Recursion

Look at the recursive calls:

```
tree(n - 1);
tree(n - 1);
```

The function calls itself two times.

So this is tree recursion.

It is called tree recursion because the calls spread out like branches of a tree.

7.3 Dry Run of tree(3)

Function call:

```
tree(3);
```

Trace:

```
tree(3)
  print 3
  tree(2)
    print 2
    tree(1)
      print 1
      tree(0) stop
      tree(0) stop
    tree(1)
      print 1
      tree(0) stop
      tree(0) stop
  tree(2)
    print 2
    tree(1)
      print 1
      tree(0) stop
      tree(0) stop
    tree(1)
      print 1
      tree(0) stop
      tree(0) stop
```

Final output:

```
3 2 1 1 2 1 1
```

7.4 Counting Calls in Tree Recursion

For this function:

```
void tree(int n) {
    if (n > 0) {
        printf("%d ", n);
        tree(n - 1);
        tree(n - 1);
    }
}
```

```
}  
}
```

For `tree(3)`, the calls are:

```
tree(3)  
tree(2), tree(2)  
tree(1), tree(1), tree(1), tree(1)  
tree(0), tree(0), tree(0), tree(0), tree(0), tree(0), tree(0), tree(0)
```

Total calls including `tree(0)` calls:

$1 + 2 + 4 + 8 = 15$ calls

Calls that print values:

$1 + 2 + 4 = 7$ printed calls

That is why the output has 7 numbers:

3 2 1 1 2 1 1

7.5 Why Tree Recursion Can Be Expensive

Tree recursion grows very fast.

For each call, the function creates two more calls.

Approximate growth:

Input n	Printed calls	Total calls including base calls
1	1	3
2	3	7
3	7	15
4	15	31
5	31	63

The pattern grows like powers of 2.

So tree recursion often has exponential time complexity.

For this example:

- Time complexity: $O(2^n)$
- Space complexity: $O(n)$ for maximum stack depth

Important point:

The total number of calls is large, but the deepest stack path is only from n down to 0 . So the maximum stack depth is $O(n)$.

8. Indirect Recursion

8.1 Meaning of Indirect Recursion

Indirect recursion happens when functions call each other in a cycle.

In simple words:

Function A does not directly call itself. Instead, A calls B, and B calls A again.

Example

```
#include <stdio.h>

void B(int n);

void A(int n) {
    if (n > 0) {
        printf("%d ", n);
        B(n - 1);
    }
}

void B(int n) {
    if (n > 1) {
        printf("%d ", n);
        A(n / 2);
    }
}
```

```
int main() {  
    A(20);  
    return 0;  
}
```

8.2 Why This Is Indirect Recursion

Look at the calls:

```
A() calls B()  
B() calls A()
```

So the recursion does not happen inside one function only.

It happens through multiple functions.

That is indirect recursion.

8.3 Why Function Prototype Is Used

In C, if function `A()` calls function `B()` before `B()` is fully written, the compiler must know that `B()` exists.

That is why we write this before `A()`:

```
void B(int n);
```

This is called a function prototype.

It tells the compiler:

`B` is a function that takes an integer and returns nothing.

Without this prototype, older C compilers may give warnings or errors.

8.4 Dry Run of A(20)

Function call:

```
A(20);
```

Trace:

```
A(20): print 20, call B(19)
B(19): print 19, call A(9)
A(9): print 9, call B(8)
B(8): print 8, call A(4)
A(4): print 4, call B(3)
B(3): print 3, call A(1)
A(1): print 1, call B(0)
B(0): stop
```

Output:

```
20 19 9 8 4 3 1
```

8.5 Warning About Indirect Recursion

Indirect recursion can be confusing because the calls move between functions.

To trace indirect recursion properly, write the function name with the value every time.

Example:

```
A(20) -> B(19) -> A(9) -> B(8) -> A(4)
```

Do not only write the numbers. Write the function names also.

9. Nested Recursion

9.1 Meaning of Nested Recursion

Nested recursion means a recursive call is used inside another recursive call.

In simple words:

The function must finish an inner recursive call before it can complete the outer recursive call.

Example

```
#include <stdio.h>

int nested(int n) {
    if (n > 100) {
        return n - 10;
    }
    return nested(nested(n + 11));
}

int main() {
    printf("%d", nested(95));
    return 0;
}
```

9.2 Why This Is Nested Recursion

Look at this line:

```
return nested(nested(n + 11));
```

There is one recursive call inside another recursive call:

```
nested( nested(n + 11) )
```

The inner call is:

```
nested(n + 11)
```

The outer call is:

```
nested(inner_result)
```

So the inner call must finish first.

That is nested recursion.

9.3 Dry Run Idea for Nested Recursion

Suppose:

```
nested(95)
```

Since `95` is not greater than `100`, this happens:

```
nested(nested(106))
```

Now the inner call must finish first:

```
nested(106)
```

Since `106 > 100`, it returns:

```
106 - 10 = 96
```

So now the original expression becomes:

```
nested(96)
```

Again, `96` is not greater than `100`, so it becomes:

```
nested(nested(107))
```

The inner call returns:

```
nested(107) = 97
```

So it becomes:

```
nested(97)
```

This process continues until the result reaches a stable answer.

Nested recursion is difficult because one recursive call controls the input of another recursive call.

9.4 Warning About Nested Recursion

Nested recursion is harder than tail, head, tree, and indirect recursion.

Why?

Because you cannot simply trace one call after another. You must solve the inner recursive call first, then use its result in the outer recursive call.

Simple rule:

For nested recursion, always solve from the inside out.

10. Full Comparison of Recursion Types

Type	How to Recognize It	Where Work Happens	Main Risk
Tail recursion	Recursive call is the final operation	Usually calling phase	Uses stack unless optimized
Head recursion	Recursive call happens before main work	Returning phase	Direct loop conversion can reverse output
Tree recursion	Function calls itself more than once	Multiple branches	Can become exponential
Indirect recursion	Multiple functions call one another in a cycle	Depends on function order	Hard to trace without function names
Nested recursion	Recursive call appears inside another recursive call	Inner call first, then outer call	Trace becomes complex quickly

11. Classification Rule

Do not classify recursion by the function name.

Classify recursion by the structure of the code.

Ask these questions:

Question 1

Is the recursive call the last operation?

If yes, it is probably tail recursion.

Question 2

Does the recursive call happen before the main work?

If yes, it is probably head recursion.

Question 3

Does the function call itself more than once?

If yes, it is tree recursion.

Question 4

Do two or more functions call each other?

If yes, it is indirect recursion.

Question 5

Is one recursive call placed inside another recursive call?

If yes, it is nested recursion.

12. Quick Identification Table

Code Pattern	Recursion Type
<code>printf(); fun(n - 1);</code>	Tail recursion if recursive call is last
<code>fun(n - 1); printf();</code>	Head recursion
<code>fun(n - 1); fun(n - 1);</code>	Tree recursion
<code>A()</code> calls <code>B()</code> , and <code>B()</code> calls <code>A()</code>	Indirect recursion
<code>fun(fun(n + 1))</code>	Nested recursion

13. Common Beginner Mistakes in Week 3

Mistake 1: Thinking Tail Recursion Means Printing Descending Only

Tail recursion is not about the output direction only.

It is about the recursive call being the last operation.

Example:

```
void tail(int n) {  
    if (n > 0) {  
        printf("%d ", n);  
        tail(n - 1);  
    }  
}
```

This is tail recursion because `tail(n - 1)` is last.

Mistake 2: Thinking Head Recursion Means Printing Ascending Only

Head recursion is not about ascending output only.

It is about the recursive call happening before the main work.

Example:

```
void head(int n) {  
    if (n > 0) {  
        head(n - 1);  
        printf("%d ", n);  
    }  
}
```

This is head recursion because the recursive call happens first.

Mistake 3: Forgetting That Tree Recursion Has More Than One Recursive Call

If a function calls itself two or more times, it is tree recursion.

Example:

```
tree(n - 1);  
tree(n - 1);
```

This creates branches, so it is tree recursion.

Mistake 4: Not Using Function Prototypes in Indirect Recursion

In C, if `A()` calls `B()` before `B()` is defined, the compiler needs a prototype.

Correct:

```
void B(int n);
```

This tells the compiler that `B()` exists.

Mistake 5: Tracing Nested Recursion Like Normal Recursion

Nested recursion must be traced from inside to outside.

Example:

```
return nested(nested(n + 11));
```

You must solve:

```
nested(n + 11)
```

first.

Then use its result in the outer call.

14. Full Practice Program for Week 3

```
#include <stdio.h>
```

```

void tail(int n) {
    if (n > 0) {
        printf("%d ", n);
        tail(n - 1);
    }
}

void head(int n) {
    if (n > 0) {
        head(n - 1);
        printf("%d ", n);
    }
}

void tree(int n) {
    if (n > 0) {
        printf("%d ", n);
        tree(n - 1);
        tree(n - 1);
    }
}

void B(int n);

void A(int n) {
    if (n > 0) {
        printf("%d ", n);
        B(n - 1);
    }
}

void B(int n) {
    if (n > 1) {
        printf("%d ", n);
        A(n / 2);
    }
}

int nested(int n) {
    if (n > 100) {
        return n - 10;
    }
    return nested(nested(n + 11));
}

int main() {
    printf("Tail recursion: ");
    tail(3);
}

```

```
printf("\nHead recursion: ");  
head(3);  
  
printf("\nTree recursion: ");  
tree(3);  
  
printf("\nIndirect recursion: ");  
A(20);  
  
printf("\nNested recursion result: %d", nested(95));  
  
return 0;  
}
```

Expected output:

```
Tail recursion: 3 2 1  
Head recursion: 1 2 3  
Tree recursion: 3 2 1 1 2 1 1  
Indirect recursion: 20 19 9 8 4 3 1  
Nested recursion result: 91
```

15. How to Study Week 3 Properly

Do not only run the code.

Use this method:

1. Read the function carefully.
 2. Find the base case.
 3. Find the recursive call or recursive calls.
 4. Check where the recursive call is placed.
 5. Check how many recursive calls are made.
 6. Decide the recursion type.
 7. Dry run with a small input like `n = 3`.
 8. Write the output before running the code.
 9. Count the number of calls for tree recursion.
 10. Try converting tail recursion into a loop.
-

16. Week 3 Practice Questions

Question 1

What type of recursion is this?

```
void fun(int n) {  
    if (n > 0) {  
        printf("%d ", n);  
        fun(n - 1);  
    }  
}
```

Answer:

Tail recursion.

Reason:

The recursive call is the last operation.

Question 2

What type of recursion is this?

```
void fun(int n) {  
    if (n > 0) {  
        fun(n - 1);  
        printf("%d ", n);  
    }  
}
```

Answer:

Head recursion.

Reason:

The recursive call happens before the main work.

Question 3

What type of recursion is this?

```
void fun(int n) {  
    if (n > 0) {  
        fun(n - 1);  
        fun(n - 1);  
    }  
}
```

Answer:

Tree recursion.

Reason:

The function calls itself more than once.

Question 4

What type of recursion is this?

```
void A(int n) {  
    if (n > 0) {  
        B(n - 1);  
    }  
}  
  
void B(int n) {  
    if (n > 0) {  
        A(n - 1);  
    }  
}
```

Answer:

Indirect recursion.

Reason:

A() calls B(), and B() calls A().

Question 5

What type of recursion is this?

```
int fun(int n) {  
    if (n > 100) {  
        return n - 10;  
    }  
    return fun(fun(n + 11));  
}
```

Answer:

Nested recursion.

Reason:

A recursive call is placed inside another recursive call.

17. Week 3 Mastery Checklist

Before moving to Week 4, you should be able to do these:

- Explain tail recursion in simple words.
 - Explain head recursion in simple words.
 - Explain tree recursion in simple words.
 - Explain indirect recursion in simple words.
 - Explain nested recursion in simple words.
 - Identify tail recursion from code.
 - Identify head recursion from code.
 - Identify tree recursion from code.
 - Identify indirect recursion from code.
 - Identify nested recursion from code.
 - Explain why tail recursion is easy to convert into a loop.
 - Explain why head recursion can reverse output if converted carelessly.
 - Count calls in a small tree recursion example.
 - Use function prototypes in indirect recursion.
 - Trace nested recursion from inside to outside.
-

18. Week 3 Final Summary

Week 3 teaches the main types of recursion.

Tail recursion happens when the recursive call is the last operation. It is easy to convert into a loop because no work is waiting after the recursive call.

Head recursion happens when the recursive call comes before the main work. The function goes down first, then performs work while returning.

Tree recursion happens when a function calls itself more than once. It can grow very fast because each call creates branches.

Indirect recursion happens when multiple functions call one another in a cycle. In C, function prototypes are important when using indirect recursion.

Nested recursion happens when a recursive call is used inside another recursive call. The inner recursive call must finish before the outer recursive call can continue.

The most important Week 3 rule is:

Do not classify recursion by the function name. Classify it by the position and number of recursive calls.

Ask these questions every time:

1. Is the recursive call last?
2. Is the recursive call before the main work?
3. Are there multiple recursive calls?
4. Are multiple functions calling each other?
5. Is a recursive call inside another recursive call?

If you can answer these questions, you can classify recursion correctly.

Week 1 taught what recursion is.

Week 2 taught how recursion uses memory.

Week 3 teaches how to recognize recursion patterns.